



Université Abdelmalek Essaâdi
Transformation digitale et intelligence
artificielle



RAPPORT DE TP

Application Web de Gestion de Produits

Architecture MVC2 avec Servlet, JSP, JSTL et Hibernate/JPA

Encadré par : Pr.Mohamed CHERRADI

Réalisé par : Asahaad Kaoutar

Table des matières

ANALYSE FONCTIONNELLE	3
1.1 Contexte du Projet	3
2.2 Fonctionnalités par acteur	3
ARCHITECTURE DU PROJET	4
2.1 Architecture MVC2.....	4
2.2 CONFIGURATION.....	4
2.3 Le flux complet d'une requête	6
2.4 Structure des packages	6
ÉTAPES DE CRÉATION DU PROJET	7
3.1 Création des modèles (Entités JPA).....	7
3.2 Couche DAO - Data Access Object	7
3.3 Couche Service.....	8
3.4 Couche Web (Servlets).....	9
3.5 Filtres de sécurité.....	11

ANALYSE FONCTIONNELLE

1.1 Contexte du Projet

Ce projet consiste à développer une application web complète de gestion des produits. L'application suit le patron d'architecture MVC2 (Model-View-Controller, version 1) basé sur les technologies Servlet, JSP et JSTL. Elle intègre un système d'authentification avec deux rôles distincts : USER (lecture seule) et ADMIN (gestion complète des produits)

2.2 Fonctionnalités par acteur

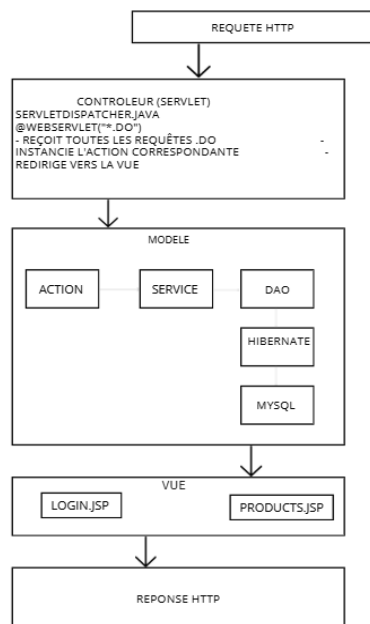
Le tableau ci-dessous résume les fonctionnalités accessibles à chaque type d'utilisateur de l'application :

Acteur	Fonctionnalites
user	- S'authentifier - Consulter la liste des produits (lecture seule)
admin	S'authentifier Consulter la liste des produits Ajouter un produit Modifier un produit Supprimer un produit

ARCHITECTURE DU PROJET

2.1 Architecture MVC2

Dans une architecture MVC2, on utilise généralement un **"Front Controller"**, qui est un point d'entrée unique pour toutes les requêtes de l'application. Ce contrôleur unique **reçoit toutes les demandes**, analyse l'URL appelée, et décide quel traitement exécuter. Pour éviter que ce contrôleur unique ne devienne trop gros et difficile à maintenir, on associe le **"Pattern Command"**. Ce patron de conception consiste à créer une petite classe séparée pour chaque action possible de l'application. La classe LoginAction s'occupe uniquement de la connexion, la classe ProductListAction s'occupe uniquement de l'affichage des produits, et ainsi de suite. Le Front Controller se contente d'instancier la bonne Action en fonction de l'URL et de l'exécuter.



2.2 CONFIGURATION

2.2.1 Configuration Maven (pom.xml)

`pom.xml` est un fichier de configuration Maven qui **gère les dépendances** du projet comme Hibernate, JPA et MySQL, et définit également les paramètres de compilation et la version Java pour l'application.

2.3.2 Configuration Hibernate (hibernate.cfg.xml)

→ "Comment se connecter à la BDD ?"

Une fois l'architecture MVC2 mise en place, il faut connecter l'application à une base de données pour stocker les informations de façon permanente. C'est là qu'intervient Hibernate, un framework dit "ORM" (Object-Relational Mapping). Sans Hibernate, le développeur devrait écrire manuellement des requêtes SQL pour chaque opération sur la base de données, puis convertir les résultats en objets Java, ce qui

génère beaucoup de code répétitif et source d'erreurs. Hibernate automatise tout ce travail. Le principe est simple : on crée des classes Java ordinaires qui représentent les données (par exemple une classe User avec des attributs nom, mot de passe et rôle), et on ajoute des annotations pour indiquer à Hibernate à quelle table et à quelles colonnes correspond chaque attribut. Une fois cette configuration effectuée, Hibernate est capable de traduire automatiquement un appel en une requête SQL

2.3.3 UTILITAIRE HibernateUtil.java)

→ "Comment créer et gérer les sessions ?"

La classe HibernateUtil est un singleton qui initialise la SessionFactory une seule fois au démarrage de l'application. La SessionFactory est l'objet central d'Hibernate : elle gère les connexions à la base de données et crée les Sessions (équivalent d'une connexion à la BDD pour une opération donnée).

```
package util;

import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
import model.User;
import model.Produit;

public class HibernateUtil {
    private static final SessionFactory sessionFactory;

    static {
        try {
            sessionFactory = new Configuration()
                .configure("hibernate.cfg.xml")
                .addAnnotatedClass(User.class)
                .addAnnotatedClass(Produit.class)
                .buildSessionFactory();
        } catch (Throwable ex) {
            System.err.println("Initial SessionFactory creation failed." + ex);
            throw new ExceptionInInitializerError(ex);
        }
    }

    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }

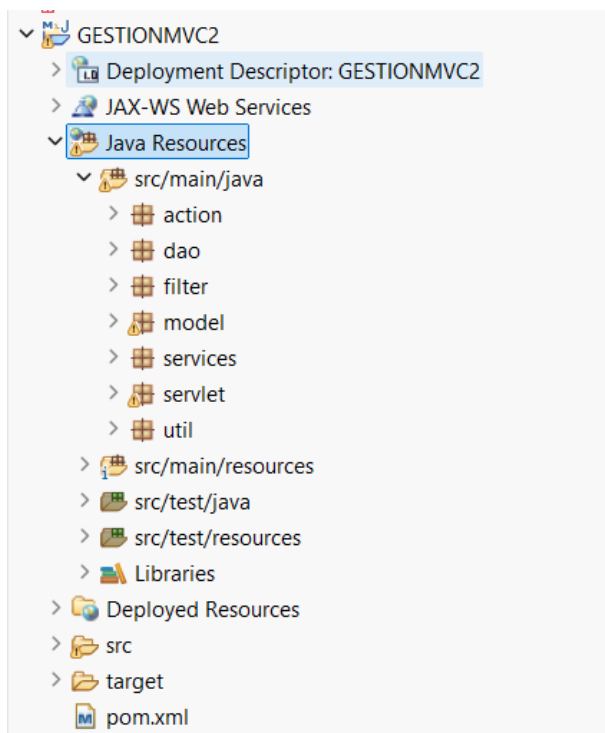
    public static void shutdown() {
        getSessionFactory().close();
    }
}
```

2.3 Le flux complet d'une requête

Lorsque tout est en place, une requête typique suit un chemin bien défini.

- L'utilisateur tape une URL ou clique sur un lien.
- Le serveur Tomcat reçoit la requête.
- Le filtre d'authentification vérifie que l'utilisateur a le droit d'accéder à cette page.
- La requête est ensuite transmise au ServletDispatcher (le Front Controller). Ce dernier analyse l'URL, sélectionne l'Action correspondante, l'exécute. L'Action appelle le Service approprié.
- Le Service appelle le DAO.
- Le DAO utilise Hibernate pour interroger la base de données.
- Les données remontent la chaîne jusqu'à l'Action. L'Action place les données dans l'objet "request" ou "session" selon leur périmètre d'utilisation.
- Enfin, l'Action retourne le nom de la page JSP à afficher. Le ServletDispatcher redirige ou transmet la requête à cette JSP, qui génère la page HTML finale. L'utilisateur voit le résultat.

2.4 Structure des packages



ÉTAPES DE CRÉATION DU PROJET

3.1 Création des modèles (Entités JPA)

Les entités JPA sont des classes Java simples (POJO) annotées pour indiquer à Hibernate comment les mapper sur les tables de la base de données. Chaque classe correspond à une table, chaque attribut à une colonne.

```
package model;
import java.io.Serializable;
import javax.persistence.*;

@Entity
@Table(name = "users")
public class User implements Serializable {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
    @Column(unique = true, nullable = false, length = 50)
    private String username;

    @Column(nullable = false, length = 100)
    private String password;

    @Column(length = 20)
    private String role;

    public User() {}

    public User(String username, String password, String role) {
        this.username = username;
        this.password = password;
        this.role = role;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
    public String getRole() { return role; }
    public void setRole(String role) { this.role = role; }
}
```

```
package model;
import java.io.Serializable;

@Entity
@Table(name = "produits")
public class Produit implements Serializable {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    @Column(nullable = false, length = 100)
    private String nom;

    @Column(nullable = false)
    private double prix;

    @Column(nullable = false)
    private int quantite;

    public Produit() {}

    public Produit(String nom, double prix, int quantite) {
        this.nom = nom;
        this.prix = prix;
        this.quantite = quantite;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getNom() { return nom; }
    public void setNom(String nom) { this.nom = nom; }
    public double getPrix() { return prix; }
    public void setPrix(double prix) { this.prix = prix; }
    public int getQuantite() { return quantite; }
    public void setQuantite(int quantite) { this.quantite = quantite; }
}
```

3.2 Couche DAO- Data Access Object

La couche DAO est responsable de l'accès à la base de données via Hibernate.

UserDAO.java

```
package dao;

import model.User;

public interface UserDAO {
    User findByUsername(String username);
    void save(User user);
}
```

ProductDAO.java

```
package dao;

import model.User;

public interface UserDAO {
    User findByUsername(String username);
    void save(User user);
}
```

UserDAOImpl.java

```

package dao;

import model.User;
import org.hibernate.Session;
import org.hibernate.query.Query;
import util.HibernateUtil;

public class UserDaoImpl implements UserDao {

    @Override
    public User findByUsername(String username) {
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            Query<User> query = session.createQuery("FROM User WHERE username = :username", User.class);
            query.setParameter("username", username);
            return query.uniqueResult();
        } catch (Exception e) {
            e.printStackTrace();
            return null;
        }
    }

    @Override
    public void save(User user) {
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            session.beginTransaction();
            session.save(user);
            session.getTransaction().commit();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

ProductDAOImpl.java

```

package dao;
import model.Produit;
import org.hibernate.Session;
import org.hibernate.Transaction;
import util.HibernateUtil;
import java.util.List;
public class ProductDAOImpl implements ProductDAO {
    @Override
    public void save(Produit product) {
        Transaction transaction = null;
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            transaction = session.beginTransaction();
            session.save(product);
            transaction.commit();
        } catch (Exception e) {
            if (transaction != null) transaction.rollback();
            e.printStackTrace();
        }
    }
    @Override
    public void update(Produit product) {
        Transaction transaction = null;
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            transaction = session.beginTransaction();
            session.update(product);
            transaction.commit();
        } catch (Exception e) {
            if (transaction != null) transaction.rollback();
            e.printStackTrace();
        }
    }
    @Override
    public void delete(int id) {
        Transaction transaction = null;
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            transaction = session.beginTransaction();
            Produit product = session.get(Produit.class, id);
            if (product != null) {
                session.delete(product);
            }
            transaction.commit();
        } catch (Exception e) {
            if (transaction != null) transaction.rollback();
            e.printStackTrace();
        }
    }
    @Override
    public Produit findById(int id) {
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            return session.get(Produit.class, id);
        }
    }
    @Override
    public List<Produit> findAll() {
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            return session.createQuery("FROM Produit", Produit.class).list();
        }
    }
}

```

3.3 Couche Service

La couche Service contient la logique métier et fait le lien entre les Actions et les DAO.

UserService.java

```
package services;

public interface UserService {
    boolean authenticate(String username, String password);
    String getRole(String username);
}
```

ProductService.java

```
package services;

import model.Produit;

public interface ProductService {
    void addProduct(Produit product);
    void updateProduct(Produit product);
    void deleteProduct(int id);
    Produit getProductById(int id);
    List<Produit> getAllProducts();
}
```

UserServiceImpl.java

```
package services;

import dao.UserDAO;

public class UserServiceImpl implements UserService {

    private UserDAO userDAO = new UserDAOImpl();

    @Override
    public boolean authenticate(String username, String password) {
        User user = userDAO.findByUsername(username);
        return user != null && user.getPassword().equals(password);
    }

    @Override
    public String getRole(String username) {
        User user = userDAO.findByUsername(username);
        return user != null ? user.getRole() : "USER";
    }
}
```

ProductServiceImpl.java

```
package services;

import dao.ProductDAO;
import dao.ProductDAOImpl;
import model.Produit;
import java.util.List;

public class ProductServiceImpl implements ProductService {

    private ProductDAO productDAO = new ProductDAOImpl();

    @Override
    public void addProduct(Produit product) {
        productDAO.save(product);
    }

    @Override
    public void updateProduct(Produit product) {
        productDAO.update(product);
    }

    @Override
    public void deleteProduct(int id) {
        productDAO.delete(id);
    }

    @Override
    public Produit getProductById(int id) {
        return productDAO.findById(id);
    }

    @Override
    public List<Produit> getAllProducts() {
        return productDAO.findAll();
    }
}
```

3.4 Couche Web (Servlets)

Le **Front Controller** est le point d'entrée unique de l'application.

ServletDispatcher.java

```

package servlet;
import java.io.IOException;
@WebServlet("*.do")
public class ServletDispatcher extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response) {}
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response) {}
    private void processRequest(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        String path = request.getServletPath();
        Action action = null;
        switch(path) {
            case "/login.do":
                action = new LoginAction();
                break;
            case "/logout.do":
                action = new LogoutAction();
                break;
            case "/products.do":
            case "/admin.do": // Les deux URLs vont à ProductListAction
                action = new ProductListAction();
                break;
            case "/addProduct.do":
                action = new AddProductAction();
                break;
            case "/deleteProduct.do":
                action = new DeleteProductAction();
                break;
            case "/editProduct.do":
                action = new EditProductAction();
                break;
            default:
                action = new LoginAction();
        }
    }

    String view = action.execute(request, response);

    if (view != null) {
        if (view.startsWith("redirect:")) {
            response.sendRedirect(view.substring(9));
        } else {request.getRequestDispatcher("/view/" + view).forward(request, response);}
    }
}

```

Explication :

- `@WebServlet("*.do")` : Capture toutes les requêtes se terminant par `.do`
- `switch(path)` : Sélectionne l'Action à exécuter selon l'URL
- `view.startsWith("redirect:")` : Si l'Action retourne "redirect", on redirige
- Sinon, on fait un forward vers la JSP correspondante

```

package action;

import javax.servlet.http.HttpServletRequest;

public interface Action {
    String execute(HttpServletRequest req, HttpServletResponse resp);
}

package action;

import javax.servlet.http.HttpServletRequest;

public class LoginAction implements Action {
    private UserService userService = new UserServiceImpl();

    @Override
    public String execute(HttpServletRequest req, HttpServletResponse resp) {
        if ("GET".equals(req.getMethod())) {
            return "login.jsp";
        }

        String username = req.getParameter("username");
        String password = req.getParameter("password");

        if (userService.authenticate(username, password)) {
            HttpSession session = req.getSession();
            session.setAttribute("user", username);
            session.setAttribute("role", userService.getRole(username));
            return "redirect:products.do";
        } else {
            req.setAttribute("error", "Login ou mot de passe incorrect");
            return "login.jsp";
        }
    }
}

```

3.5 Filtres de sécurité

Les filtres Java EE s'exécutent AVANT que la requête n'atteigne le Servlet. Ils permettent d'intercepter et d'examiner chaque requête pour appliquer des contrôles de sécurité de manière centralisée, sans avoir à répéter le code dans chaque Servlet.

AuthFilter.java :

```

package filter;

import java.io.IOException;

@WebFilter({"*.do", "*.jsp"})
public class AuthFilter implements Filter {

    @Override
    public void init(FilterConfig filterConfig) throws ServletException {}

    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
        throws IOException, ServletException {

        HttpServletRequest req = (HttpServletRequest) request;
        HttpServletResponse res = (HttpServletResponse) response;
        HttpSession session = req.getSession(false);

        String path = req.getServletPath();

        // Pages publiques
        if (path.equals("/login.do") || path.equals("/view/login.jsp")) {
            chain.doFilter(request, response);
            return;
        }

        // Vérifier authentification
        if (session == null || session.getAttribute("user") == null) {
            res.sendRedirect(req.getContextPath() + "/login.do");
            return;
        }

        chain.doFilter(request, response);
    }

    @Override
    public void destroy() {}
}

```

RoleFilter.java

```

package filter;

import java.io.IOException;

@WebFilter("/admin/*")
public class RoleFilter implements Filter {

    @Override
    public void init(FilterConfig filterConfig) throws ServletException {}

    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
        throws IOException, ServletException {

        HttpServletRequest req = (HttpServletRequest) request;
        HttpServletResponse res = (HttpServletResponse) response;
        HttpSession session = req.getSession(false);

        String role = (String) session.getAttribute("role");

        // Vérifier si l'utilisateur est ADMIN
        if (role == null || !role.equals("ADMIN")) {
            res.sendRedirect(req.getContextPath() + "/login.do");
            return;
        }

        chain.doFilter(request, response);
    }

    @Override
    public void destroy() {}
}

```

```

package util;

import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
import model.User;
import model.Produit;

public class HibernateUtil {
    private static final SessionFactory sessionFactory;

    static {
        try {
            sessionFactory = new Configuration()
                .configure("hibernate.cfg.xml")
                .addAnnotatedClass(User.class)
                .addAnnotatedClass(Produit.class)
                .buildSessionFactory();
        } catch (Throwable ex) {
            System.err.println("Initial SessionFactory creation failed." + ex);
            throw new ExceptionInInitializerError(ex);
        }
    }

    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }

    public static void shutdown() {
        getSessionFactory().close();
    }
}

```

Démonstration (captures) :

1- Page de connexion (login.jsp)

Connexion

2- Liste des produits - Vue Utilisateur

Les Produits					admin (ADMIN)	Déconnexion
Ajouter						
ID	Nom	Prix	Quantité	Actions		
2	Produit 2	200.0	50	Modifier	Supprimer	

3- Liste des produits - Vue Admin

Les Produits					user (USER)	Déconnexion
ID	Nom	Prix	Quantité			
2	Produit 2	200.0	50			

— Fin du rapport —